정세진 Backend Developer

문제를 분석하고 구조를 개선하는 백엔드 개발자

안정적인 데이터 처리와 서비스 운영을 고려하며 백엔드 서비스를 개발합니다.

GitHub
https://github.com/sejinzx

Email
jeongsejin789@gmail.com

블로그
https://vivid-reminiscences.tistory.com/

보유기술

Core

Java | Spring Boot | MySQL

Experienced

Apache Kafka | Spring Security | Docker | Kubernetes | GitHub Actions | AWS | NHN Cloud | React

Collaboration & Tools

GitHub | Notion | Slack

PROJECT 01

클래스룸

프로젝트 개요

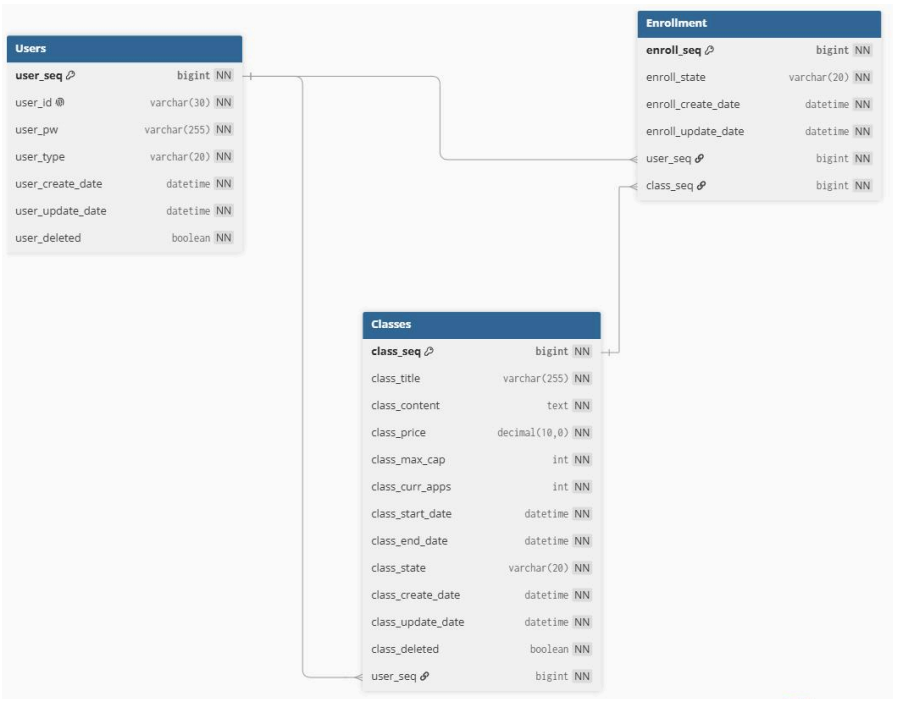
사용자가 강의를 조회하고 신청·취소할 수 있는 웹 서비스입니다.
다수의 사용자가 동시에 신청하더라도 정원을 초과하지 않고 데이터 정합성을 유지하도록 수강 신청 구조를 개선했습니다.

- 기간: 2026.05~09
- 형태: 개인 프로젝트
- 기술스택: Java, Spring Boot, MySQL, Apache Kafka, Docker, GitHub Actions, AWS, JMeter, Testcontainers
- GitHub: https://github.com/sejinzx/enrollment_system
- Web: http://13.209.75.120:3000

주요 구현

- Spring Security + JWT 기반 인증/인가
- Kafka 기반 비동기 수강 신청 처리
- 조건부 UPDATE를 통한 동시성 제어
- UNIQUE 제약을 통한 중복 신청 방지
- Kafka 재시도 및 DLT 기반 장애 처리
- Testcontainers 기반 통합 테스트
- GitHub Actions 기반 CI/CD 및 EC2 자동 배포

ERD



조건부 UPDATE를 적용한 동시 요청 데드락 해결

문제 및 원인

비관적 락 적용 후 동시 요청이 증가하면서 동일 강의 데이터에 대한 락 경쟁과 데드락이 발생했습니다.

조건부 UPDATE 적용

기존의 조회 → 정원 확인 → 신청 인원 증가 구조를 제거하고, 정원 확인과 신청 인원 증가를 하나의 조건부 UPDATE 쿼리로 처리했습니다.

```
UPDATE classes
SET class_curr_apps = class_curr_apps + 1
WHERE class_seq = ?
   AND class_curr_apps < class_max_cap;
```

- 또한 (user_seq, class_seq) UNIQUE 제약조건을 적용해 동시 요청으로 발생할 수 있는 중복 신청을 방지했습니다.

동시성 및 데이터 정합성 검증

- JUnit 기반 동시성 테스트를 통해 여러 요청이 동시에 발생하는 상황에서 수강 신청 로직이 정상적으로 처리되는지 검증했습니다.
- JMeter로 1,000건의 동시 요청을 발생시켜 정원 초과 0건, 중복 신청 0건을 확인하고, 최종 신청 인원과 수강 신청 데이터 수가 일치하는 것을 검증했습니다.

PROJECT 01

클래스룸

Kafka 기반 비동기 수강 신청 처리

- 수강 신청 요청과 실제 처리 로직을 분리하기 위해 Kafka 기반 Producer–Consumer 구조를 적용했습니다.
- API에서 수강 신청 이벤트를 enrollment-request Topic에 발행하고, Consumer가 메시지를 수신해 정원 확인 및 신청 데이터를 저장하도록 구성했습니다.

처리 흐름

수강 신청 요청 → Producer → Kafka Topic → Consumer → 신청 처리 → DB 반영

메시지 처리 신뢰성 강화

Consumer에서 비즈니스 예외와 시스템 예외를 구분해 처리했습니다.

- 중복 신청·정원 초과 등의 비즈니스 예외는 재시도하지 않고 종료
- DB 장애 등 시스템 예외는 1초 간격으로 최대 2회 재시도
- 재시도 후에도 실패한 메시지는 enrollment-request-dlt로 이동
- 동일한 메시지가 재전달되더라도 UNIQUE 제약과 트랜잭션을 통해 신청 데이터와 신청 인원의 중복 반영 방지

Kafka 통합 테스트

Testcontainers로 Kafka와 MySQL 환경을 구성하고 실제 Producer–Consumer 흐름을 기준으로 다음 시나리오를 검증했습니다.

- 메시지 발행 → Consumer 수신 → DB 정상 반영
- 일시적인 DB 장애 발생 → 재시도 후 처리
- 반복적인 처리 실패 → DLT 이동
- 동일 메시지 중복 전달 → 신청 데이터 1건, 신청 인원 1명만 반영

PROJECT 02

Talk-kit

프로젝트 개요

AI 대본 생성, 음성 녹음, 프로젝트 관리 및 커뮤니티 피드백 기능을 제공하는 AI 기반 발표 연습 플랫폼입니다.
Spring Cloud Gateway와 Eureka를 활용해 기능별 서비스를 분리한 MSA 구조로 개발했습니다.
프로젝트 진행 중 팀 구성 변경으로 역할이 재조정된 상황에서 남아 있던 백엔드 기능과 서비스 간 연동 오류를 보완해 프로젝트 마무리했습니다.

- 기간: 2024.06~10
- 인원: Backend 3 / Frontend 2
- 담당 역할: 유저 서비스, 커뮤니티 서비스 개발
- 기술스택: Java, Spring Boot, Spring Cloud Gateway, Eureka, MySQL, Redis, JWT, Docker, Kubernetes, NHN Cloud
- GitHub: https://github.com/orgs/Talk-Kit/repositories

담당 역할

유저 서비스

- 사용자 CRUD API
- Redis TTL 기반 이메일 인증
- 인증번호 저장 및 만료 처리

커뮤니티 서비스

- 게시글 CRUD API
- 게시글 좋아요 및 신고
- 게시글과 프로젝트 파일 통합 조회 API

Feign Client를 활용한 서비스 간 데이터 조회

문제 및 원인

게시글 조회 API에서 프로젝트 파일 정보도 함께 제공해야 했지만, 파일 정보는 프로젝트 서비스에서 관리하고 있어 커뮤니티 서비스에서 직접 조회할 수 없었습니다.

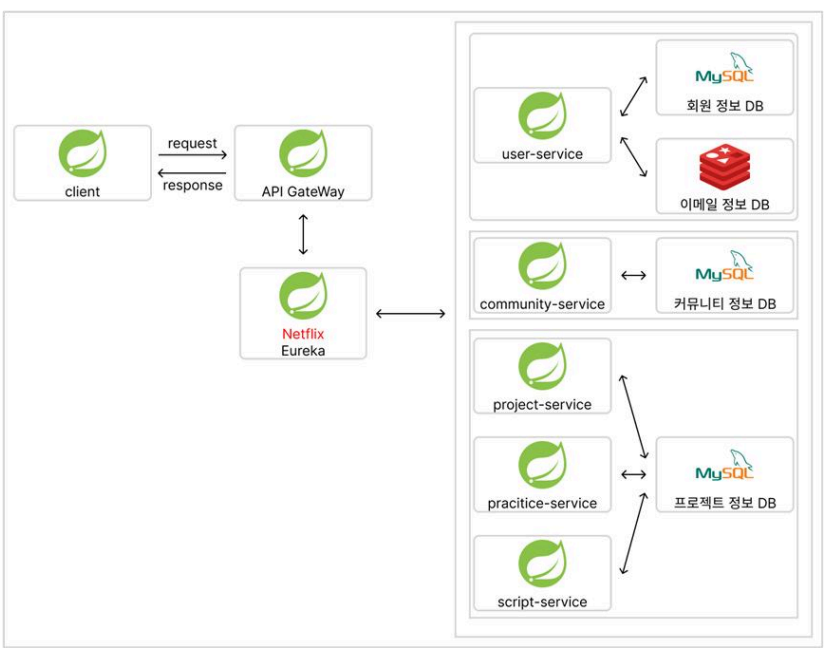
서비스 간 API 호출 구조 적용

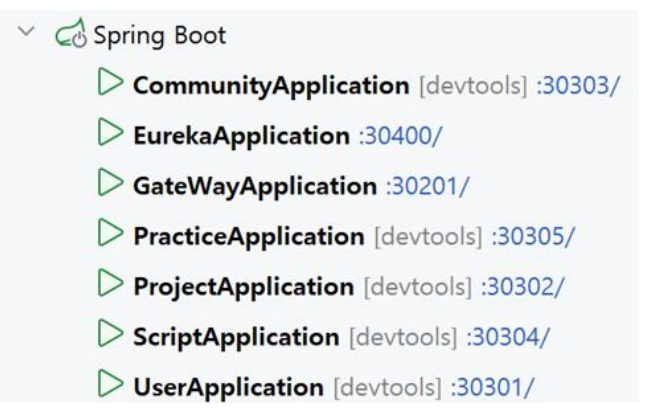
- Feign Client를 적용해 커뮤니티 서비스에서 프로젝트 서비스의 파일 조회 API를 호출하도록 구성했습니다.
- 조회한 게시글과 파일 데이터를 조합해 클라이언트가 필요한 정보를 하나의 응답으로 제공했습니다.

결과

- Feign Client를 통해 커뮤니티 서비스와 프로젝트 서비스를 연동하고, 각 서비스의 DB를 직접 참조하지 않고도 게시글과 프로젝트 파일 정보를 하나의 응답으로 제공했습니다.

시스템 아키텍처





PROJECT 03

일정 관리 시스템 (Scheduling)

프로젝트 개요

React와 Spring Boot 기반의 일정 및 Todo 관리 웹 애플리케이션입니다.
애플리케이션 개발뿐 아니라 Spring Boot, React, MySQL을 컨테이너 환경으로 구성하고 Jenkins를 이용해 빌드 및 실행 과정을 자동화했습니다.

- 기간: 2026.04~06
- 형태: 개인 프로젝트
- 기술스택: Java, Spring Boot, React, MySQL, Docker, Jenkins
- GitHub: https://github.com/sejinzx/scheduling

주요 구현

- Spring Boot / React / MySQL Docker 컨테이너 환경 구성
- Docker Compose 기반 서비스 의존성 및 네트워크 구성
- Backend Multi-stage Docker Build 적용
- Jenkins Pipeline 기반 애플리케이션 재빌드·재기동 자동화

Docker 기반 실행환경 구성 및 빌드·실행 자동화

컨테이너 환경에서 발생한 서비스 간 통신 문제 해결

- 초기에는 React가 Spring Boot API를 localhost:8080으로 호출하도록 설정되어 있었습니다.
- 각 애플리케이션을 Docker 컨테이너로 분리하면서 localhost는 백엔드가 아닌 React 컨테이너 자신을 가리키게 되었고, 프론트엔드에서 백엔드 API에 접근할 수 없는 문제가 발생했습니다.
- Spring Boot, React, MySQL을 동일한 Docker Network에 구성하고 API 요청 대상을 localhost에서 백엔드 서비스명인 spring-app:8080으로 변경했습니다. 이를 통해 호스트 주소에 의존하지 않고 Docker 내부 DNS를 이용해 컨테이너 간 통신하도록 구조를 변경했습니다.

Multi-stage Build를 이용한 백엔드 빌드 구조 개선

- 기존에는 빌드된 JAR 파일이 존재해야 Docker 이미지를 생성할 수 있는 구조였습니다.
- 이를 Gradle Builder 단계와 애플리케이션 실행 단계로 분리한 Multi-stage Build 방식으로 변경했습니다. Docker 이미지 생성 과정에서 소스 코드를 직접 빌드하고, 최종 실행 단계에는 생성된 JAR 파일만 전달하도록 구성해 빌드 과정과 실행 환경을 분리했습니다.

Jenkins Pipeline을 이용한 실행 과정 자동화

- 코드 변경 이후 저장소 갱신과 Docker 이미지 재빌드, 컨테이너 재실행을 반복적으로 직접 수행해야 했습니다.
- Jenkins Pipeline을 구성해 저장소의 main 브랜치를 가져온 뒤 `docker compose down` 과 `docker compose up --build -d` 가 순차적으로 실행되도록 구성했습니다.
- 또한 Jenkins 실행 환경에서도 Docker 명령을 사용할 수 있도록 Jenkins 이미지를 별도로 구성해 Docker 실행환경과 연결했습니다.

결과

- Docker 내부 DNS를 이용해 분리된 컨테이너 간 통신 문제 해결
- Multi-stage Build를 적용해 백엔드 빌드 단계와 실행 환경 분리
- Jenkins Pipeline을 통해 저장소 갱신부터 이미지 재빌드·컨테이너 재기동까지 반복 작업 자동화

PROJECT 04

Green-Us

프로젝트 개요

Green-Us는 환경 보호 활동을 등록하고 진행 상황과 리뷰를 공유하는 플랫폼입니다.
Kotlin 기반 Android 애플리케이션의 주요 화면을 개발하고, 이후 Spring Boot 백엔드의 리뷰·사용자·참여 현황 기능 개선에도 참여했습니다.

- 기간: 2024.03 ~ 11
- 인원: Frontend 2, Backend 2
- 기술: Kotlin, Spring Boot, MySQL, Android, Firebase
- GitHub: https://github.com/green-us-2024/green-us

담당역할

Android 개발

- 로그인·회원가입 및 계정 찾기 화면
- 마이페이지·프로필·그리닝 화면
- 리뷰·공지사항·포인트 목록 및 Adapter
- 프로필 이미지 처리 및 화면 전환 개선

백엔드 개발

- 리뷰 조회·수정 및 최신순 정렬 기능 개선
- 사용자 정보 수정 기능 개선
- 참여 완료 활동 조회 및 진척도 로직 개선

참여 상태에 맞는 데이터 조회 API 개선

문제 및 원인

- 기존 참여 조회 기능은 사용자의 전체 참여 내역을 반환해, 화면에서 진행 중인 활동과 완료된 활동을 구분해 사용하기 어려웠습니다.

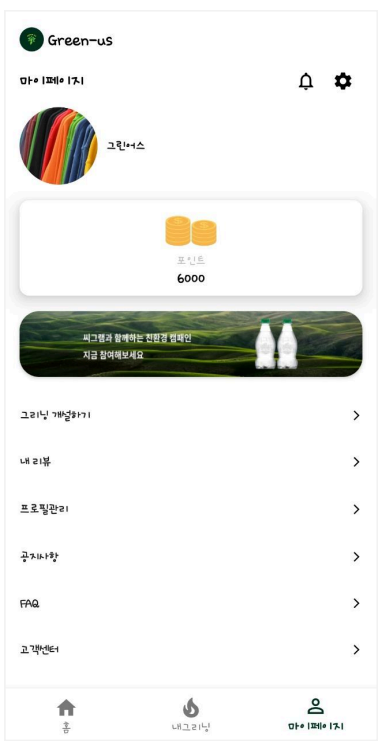
조회 구조 개선

- 행 중인 참여만 조회할 수 있도록 pComplete = 'N' 조건 적용
- Repository–Service–Controller 계층에 상태별 조회 기능 추가
- 참여 ID를 기준으로 연결된 Greening 정보를 조회하도록 기능 보완

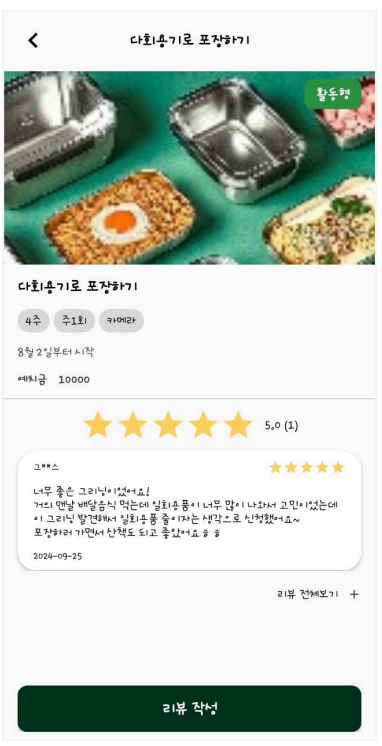
결과

- 참여 상태별 데이터 조회 기능을 분리하고, 참여 ID를 기준으로 연결된 Greening 정보를 조회하도록 개선해 화면에서 필요한 참여 정보를 구분해 사용할 수 있도록 했습니다.

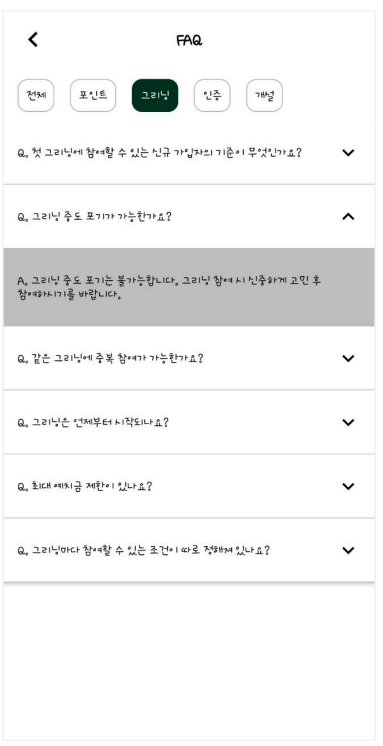
구현 화면



마이페이지



리뷰



FAQ